Angular · Spring Boot

RAG - Acreditaciones

Sistema de gestión documental con IA

2º DAW

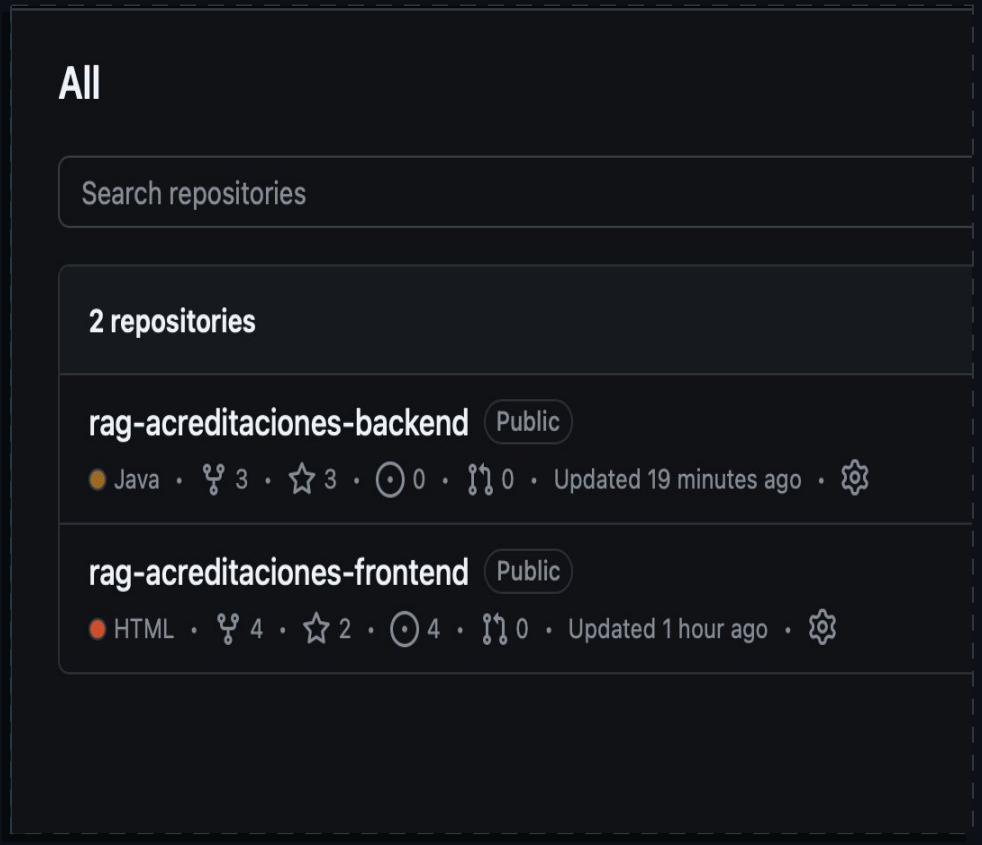
Moha
Patryk
Richart
Yordan

Proyecto
Grupo 1
2026

GitHub

INFO 1/4

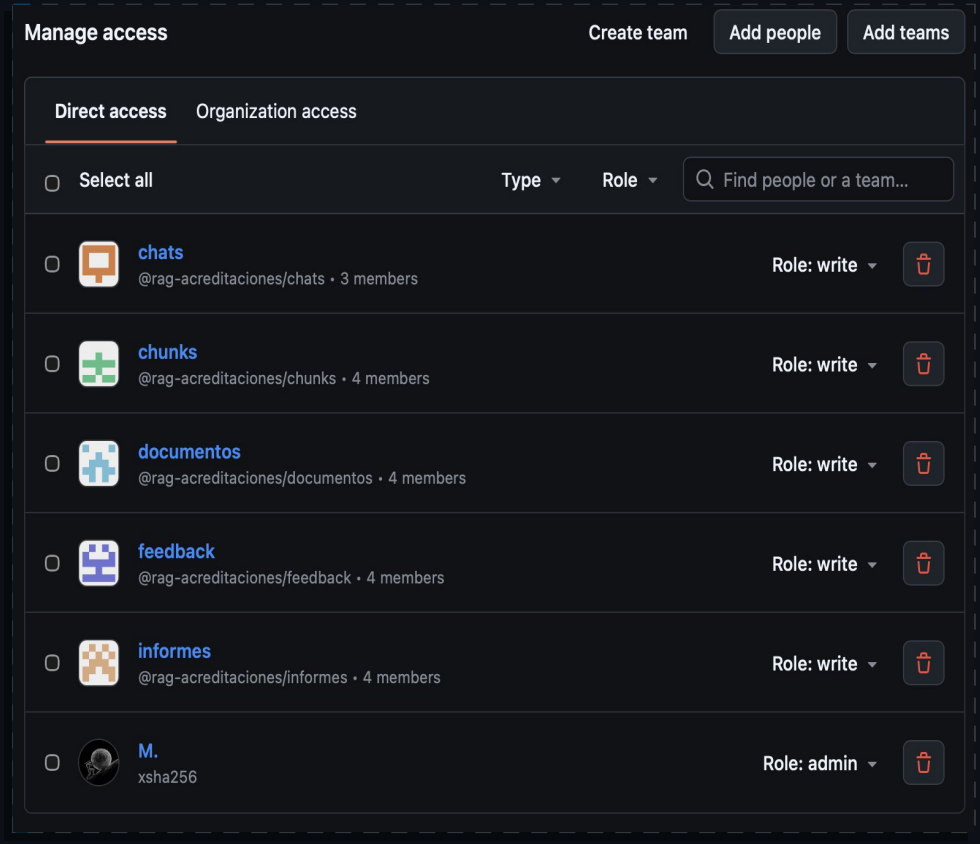
rag-acreditaciones



GitHub

INFO 2/4

Equipos



Backend

Controller

1. El filtro legacy: filter=seccionId:IGUAL:1
2. Filtros sueltos: ?seccionId=1&estado=PROCESADO

```java
List<String> allFilters = new ArrayList<>();
if (filter != null) {
    for (String f : filter) {
        allFilters.add(f);
    }
}
if (seccionId != null) {
    allFilters.add("seccionId:IGUAL:" + seccionId);
}
if (estado != null) {
    allFilters.add("estado:IGUAL:" + estado);
}
if (nombre != null) {
    allFilters.add("nombre:CONTIENE:" + nombre);
}
if (subidoPor != null) {
    allFilters.add("subidoPor:CONTIENE:" + subidoPor);
}
if (fechaDesde != null) {
    allFilters.add("fechaSubida:MAYOR_QUE:" + fechaDesde);
}
if (fechaHasta != null) {
    allFilters.add("fechaSubida:MENOR_QUE:" + fechaHasta);
}
```

Backend

ServiceImpl

1. Filtros mal formados o nulos
2. Specification para componer filtros
3. Ocultar registros eliminados

```java
if (val == null || "null".equalsIgnoreCase(val) || val.isBlank()) {
    continue;
}
```

```java
} else if ("fechaSubida".equals(attr)) {
    LocalDateTime dateVal = val.length() <= 10
            ? LocalDate.parse(val).atStartOfDay()
            : LocalDateTime.parse(val);
    spec = spec.and((root, query, cb) -> {
        switch (filtro.getOperacion()) {
            case MAYOR_QUE: return cb.greaterThanOrEqualTo(root.get(attributeName: "fechaSubida"), dateVal);
            case MENOR_QUE: return cb.lessThanOrEqualTo(root.get(attributeName: "fechaSubida"), dateVal);
            default: return cb.equal(root.get(attributeName: "fechaSubida"), dateVal);
        }
    });
```

```java
if (!tieneFiltoEstado) {
    spec = spec.and((root, query, cb) attributeName:
        -> cb.notEqual(root.get("estado"), EstadoDocumento.ELIMINADO));
}
```

Modelo de datos Db

1. DocumentoDb registra cada archivo subido
2. Guarda nombre, tipo, tamaño, autor y fecha y metadatos para el componente “preview”
3. Estado: pendiente, procesado, con error etc.
4. SeccionTematicaDb agrupa docs por tema
5. Cada sección tiene nombre, descripcion y Codigo de color
6. Validaciones impiden guardar datos corruptos

```java
@Data
@NoArgsConstructor
@AllArgsConstructor
@Entity
@Table(name = "documentos")
public class DocumentoDb {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    @Size(max = 255)
    @Column(name = "nombre_fichero", nullable = false)
    private String nombreFichero;

    @Size(max = 100)
    @Column(name = "content_type")
    private String contentType;

    @Column(name = "size_bytes")
    private Long sizeBytes;

    @Size(max = 500)
    private String descripcion;

    @Column(name = "base64_contenido", columnDefinition = "TEXT")
    private String base64Contenido;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "seccion_tematica_id")
    private SeccionTematicaDb seccionTematica;

    @NotBlank
    @Column(name = "subido_por", nullable = false)
    private String subidoPor;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private EstadoDocumento estado = EstadoDocumento.PENDIENTE;

    @Column(name = "fecha_subida", nullable = false)
    private LocalDateTime fechaSubida = LocalDateTime.now();
}
```

```java
@Data
@NoArgsConstructor
@AllArgsConstructor
@Entity
@Table(name = "secciones_tematicas")
public class SeccionTematicaDb {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    @Size(max = 100)
    @Column(nullable = false, unique = true)
    private String nombre;

    @Size(max = 500)
    private String descripcion;

    @Size(max = 7)
    @Column(name = "codigo_color")
    private String codigoColor;
}
```

```java
public enum EstadoDocumento {
    PENDIENTE,
    PROCESANDO,
    PROCESADO,
    ERROR,
    ELIMINADO
}
```

Transferencia de datos DTO

1. UploadDTO: recoge los datos al subir un PDF
2. ResponseDTO: lista sin contenido pesado
3. DetailDTO: incluye el PDF al ver el detalle
4. Los listados no cargan archivos en Base64
5. Cada vista recibe exactamente lo que necesita

```java
public record DocumentoUploadDTO(

        @NotNull(message = "La seccion tematica es obligatoria") Long seccionTematicaId,

        @Size(max = 500, message = "La descripcion no puede superar 500 caracteres") String descripcion) {
}
```

```java
public record DocumentoResponseDTO(
        Long id,
        String nombreFichero,
        String contentType,
        Long sizeBytes,
        String descripcion,
        Long seccionTematicaId,
        String seccionTematicaNombre,
        String subidoPor,
        String estado,
        LocalDateTime fechaSubida) {
}
```

```java
public record DocumentoDetailDTO(
        Long id,
        String nombreFichero,
        String contentType,
        Long sizeBytes,
        String descripcion,
        Long seccionTematicaId,
        String seccionTematicaNombre,
        String subidoPor,
        String estado,
        LocalDateTime fechaSubida,
        String base64Contenido) {
}
```

```java
public record DocumentoFilterDTO(
        String nombre,
        Long seccionId,
        String subidoPor,
        LocalDateTime fechaDesde,
        LocalDateTime fechaHasta,
        String estado) {
}
```

Repository

1. Repository extiende de JpaRepository
2. Repository proporcionado por Spring Data
3. Spring Data gestiona el acceso a la BD
4. Uso de JpaSpecificationExecutor
5. Flexible y eficiente para cualquier búsqueda

```
public interface SeccionTematicaRepository extends JpaRepository<SeccionTematicaDb, Long> {

}
```

```
public interface DocumentoRepository extends JpaRepository<DocumentoDb, Long>, JpaSpecificationExecutor<DocumentoDb> {

}
```

Servicios Frontend

1. Los servicios gestionan toda la API
2. Filtros avanzados con JPA enviados como parámetros para permitir búsquedas más dinámicas
3. Login y logout trabajan con localStorage

```
getDocumentos(params: DocumentoListParams = {}): Observable<PaginaResponse<Documento>> {
  let httpParams = new HttpParams()
    .set('page', params.page ?? 0)
    .set('size', params.size ?? 10)
    .set('sort', params.sort ?? 'fechaSubida,desc');

  const filtro = params.filtros ?? {};
  if (filtro.nombre)     httpParams = httpParams.append('nombre', filtro.nombre);
  if (filtro.seccionId)  httpParams = httpParams.append('seccionId', filtro.seccionId.toString());
  if (filtro.subidoPor)  httpParams = httpParams.append('subidoPor', filtro.subidoPor);
  if (filtro.estado)     httpParams = httpParams.append('estado', filtro.estado);
  if (filtro.fechaDesde) httpParams = httpParams.append('fechaDesde', filtro.fechaDesde);
  if (filtro.fechaHasta) httpParams = httpParams.append('fechaHasta', filtro.fechaHasta);

  return this.http.get<PaginaResponse<Documento>>(`${API}/documentos`, { params: httpParams });
}
```

```
logout(): void {
    localStorage.removeItem(this.TOKEN_KEY);
    localStorage.removeItem(this.NICKNAME_KEY);
    this.router.navigate(['/login']);
  }
```

```
login(credentials: LoginRequest): Observable<JwtResponse> {
    return this.http
      .post<JwtResponse>(`${environment.apiUrl}/auth/login`, credentials)
      .pipe(
        tap((res) => {
          localStorage.setItem(this.TOKEN_KEY, res.token);
          localStorage.setItem(this.NICKNAME_KEY, res.nickname);
        })
      );
```

Modelos Frontend

1. Modelos/Interfaces para cada apartado
2. Cada Interface con sus propiedades
3. Todo centralizado en un documento para poder importar más facilmente

```typescript
// — Sección temática ————————————————————————————————
export interface SeccionTematica {
  id: number;
  nombre: string;
  descripcion: string;
  color: string;
}

// — Estados posibles de un documento ————————————————
export type DocumentoEstado =
  | 'PENDIENTE'
  | 'PROCESANDO'
  | 'PROCESADO'
  | 'ERROR'
  | 'ELIMINADO';

// — Documento (respuesta del backend - DocumentoResponseDTO) ————————
export interface Documento {
  id: number;
  nombreFichero: string;
  descripcion: string;
  fechaSubida: string;
  sizeBytes: number;
  estado: DocumentoEstado;
  subidoPor: string;
  seccionTematicaId: number;
  seccionTematicaNombre: string;
  contentType: string;
  numChunks?: number;
}
```

```typescript
// — Preview del PDF (base64 para iframe inline - DocumentoDetailDTO del backend) —
export interface DocumentoPreview {
  id: number;
  nombreFichero: string;
  base64Contenido: string;
  contentType: string;
  descripcion: string;
  fechaSubida: string;
  sizeBytes: number;
  estado: DocumentoEstado;
  subidoPor: string;
  seccionTematicaId: number;
  seccionTematicaNombre: string;
}

// — Payload de subida (multipart) ————————————————————
export interface DocumentoUploadMetadata {
  seccionTematicaId: number;
  descripcion: string;
}

// — Filtros para el listado paginado ————————————————
export interface DocumentoFiltros {
  nombre?: string;
  seccionId?: number;
  subidoPor?: string;
  fechaDesde?: string;
  fechaHasta?: string;
  estado?: DocumentoEstado;
}

// — Respuesta paginada genérica (PaginaResponse del backend) ————————
export interface PaginaResponse<T> {
  number: number;
  size: number;
  totalElements: number;
  totalPages: number;
  content: T[];
  listaFiltros: { atributo: string; operacion: string; valor: unknown }[];
  listaOrdenaciones: string[];
}
```

Interceptores y Guards

1. HttpInterceptorFn para inyectar el token Bearer en todas las peticiones al backend
2. Uso de CanActivateFn con createUrlTree para Proteger rutas privadas
3. Garantizar que no se pueda acceder a los menús sin autorizacion

```fn
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = localStorage.getItem('auth_token');

  if (token) {
    const authReq = req.clone({
      setHeaders: { Authorization: `Bearer ${token}` },
    });
    return next(authReq);
  }

  return next(req);
};
```

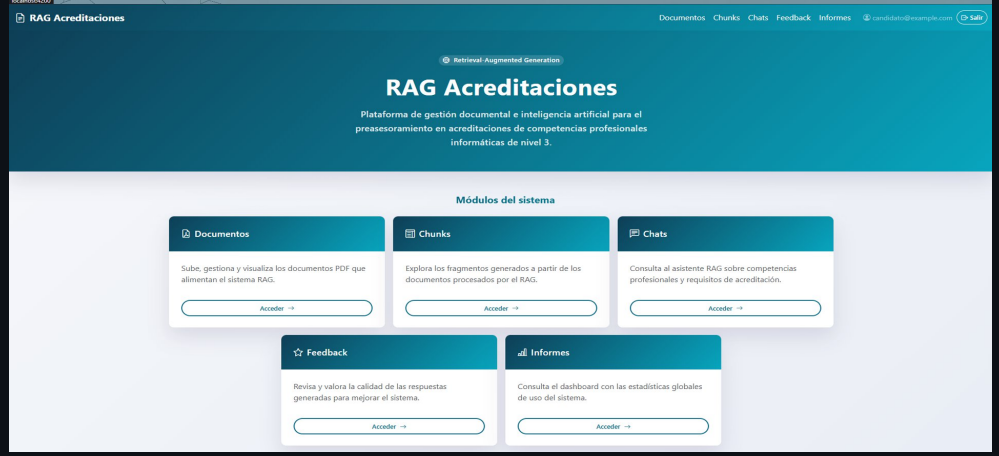
```fn
export const authGuard: CanActivateFn = () => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.isLoggedIn()) {
    return true;
  }

  return router.createUrlTree(['/login']);
};
```

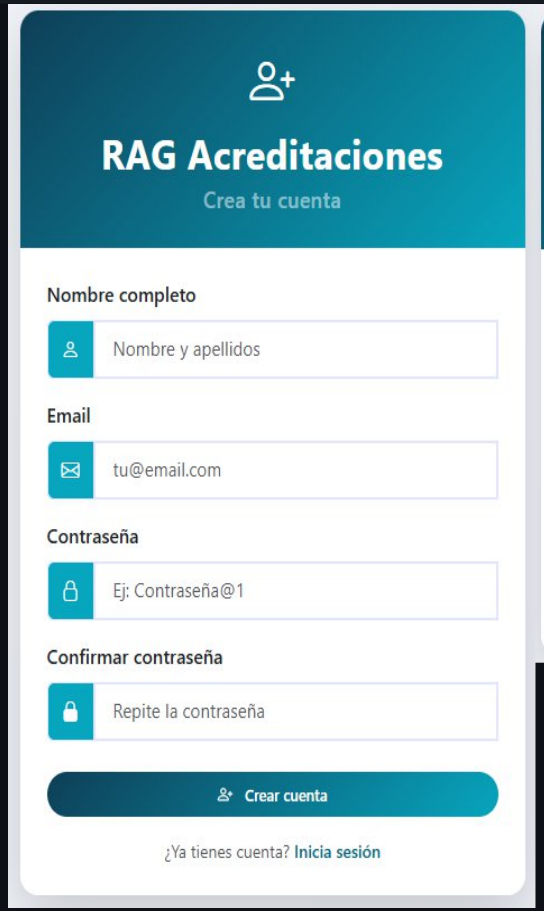
Acceso y Home-Page

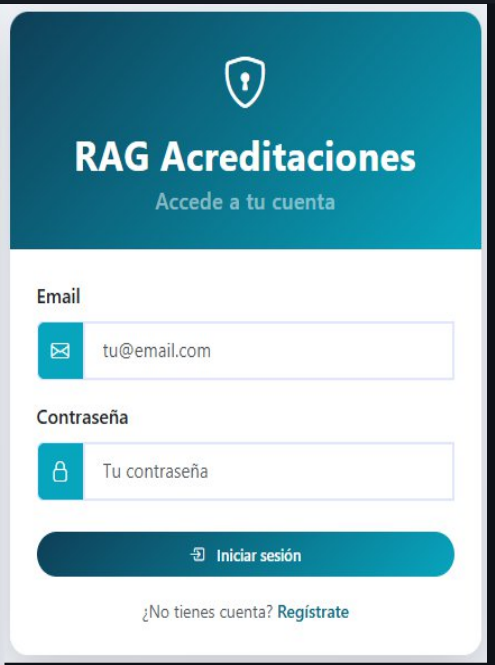
1. AuthService controla la sesión del usuario
2. El token JWT persiste en el navegador
3. Cada petición incluye el token automáticamente
4. Las rutas privadas requieren estar autenticado
5. Sin login, el guard bloquea el acceso
6. NavBar y Home cambian según la sesión



Autenticación y Registro

1. Formularios de login y registro
2. Validaciones en todos los campos
3. La interfaz responde en tiempo real
4. Sin recargas de página innecesarias
5. Enlaces dinámicos entre formularios

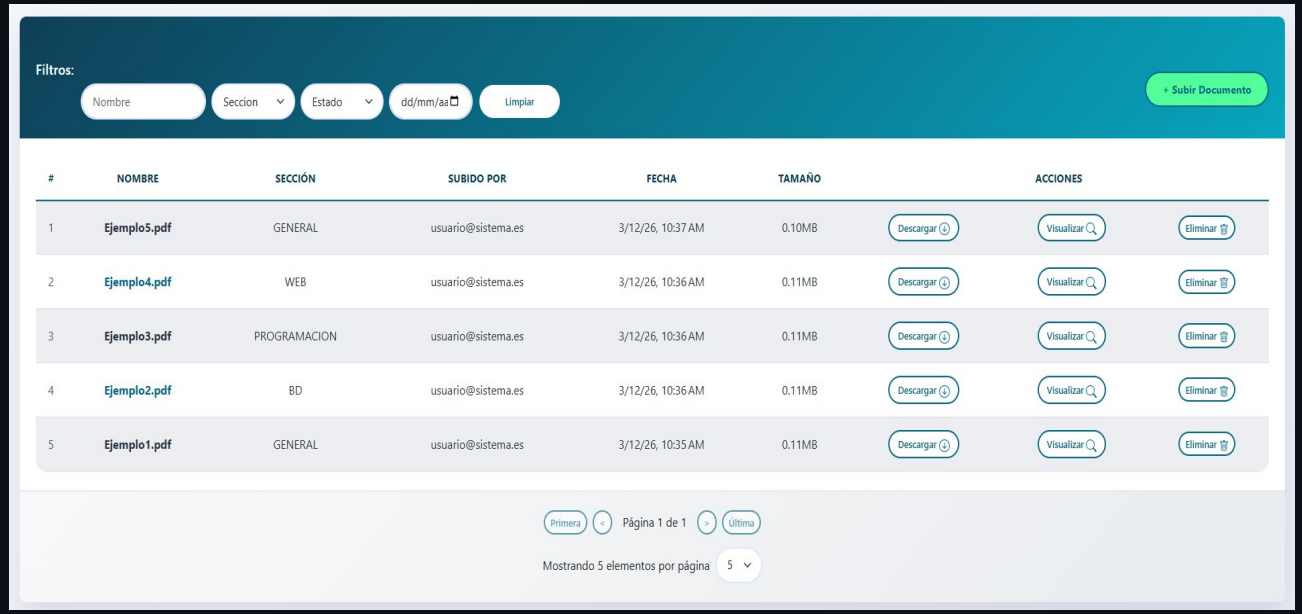




Componente Tabla de documentos

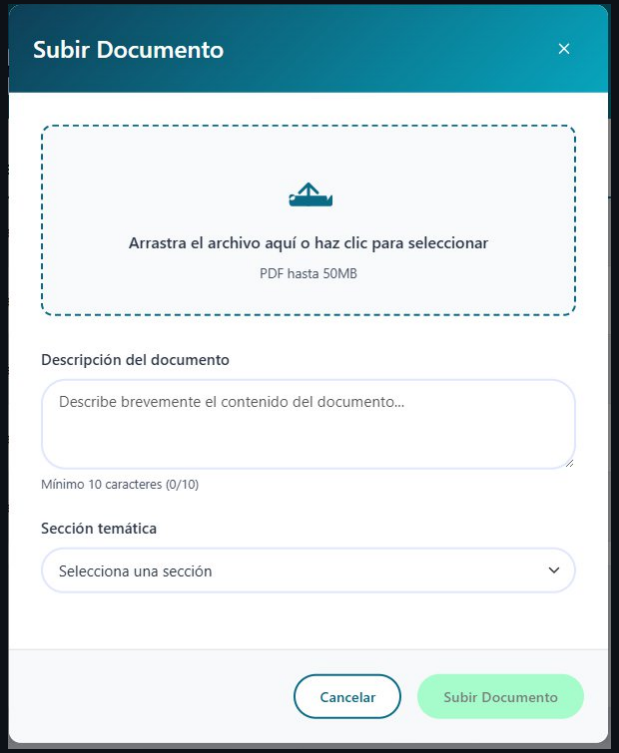
FRONTEND DOC 1 / 3

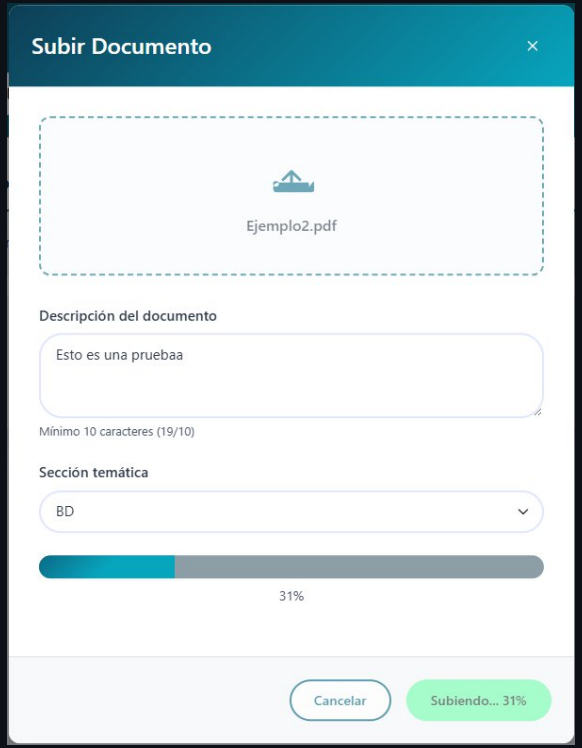
- Cada fila representa un documento subido
- Filtros: nombre, estado, fecha y sección
- Acciones: Descargar, Previsualizar, Eliminar
- Estado reactivo con Angular Signals
- Al eliminar, la fila se transforma en confirmación



Componente Modal de subida

- Modal para subir PDFs con metadatos
- Soporta arrastrar y soltar archivos
- Valida tipo, extensión y tamaño (máx. 50 MB)
- Barra de progreso durante la subida
- Notifica al componente padre al terminar de subir





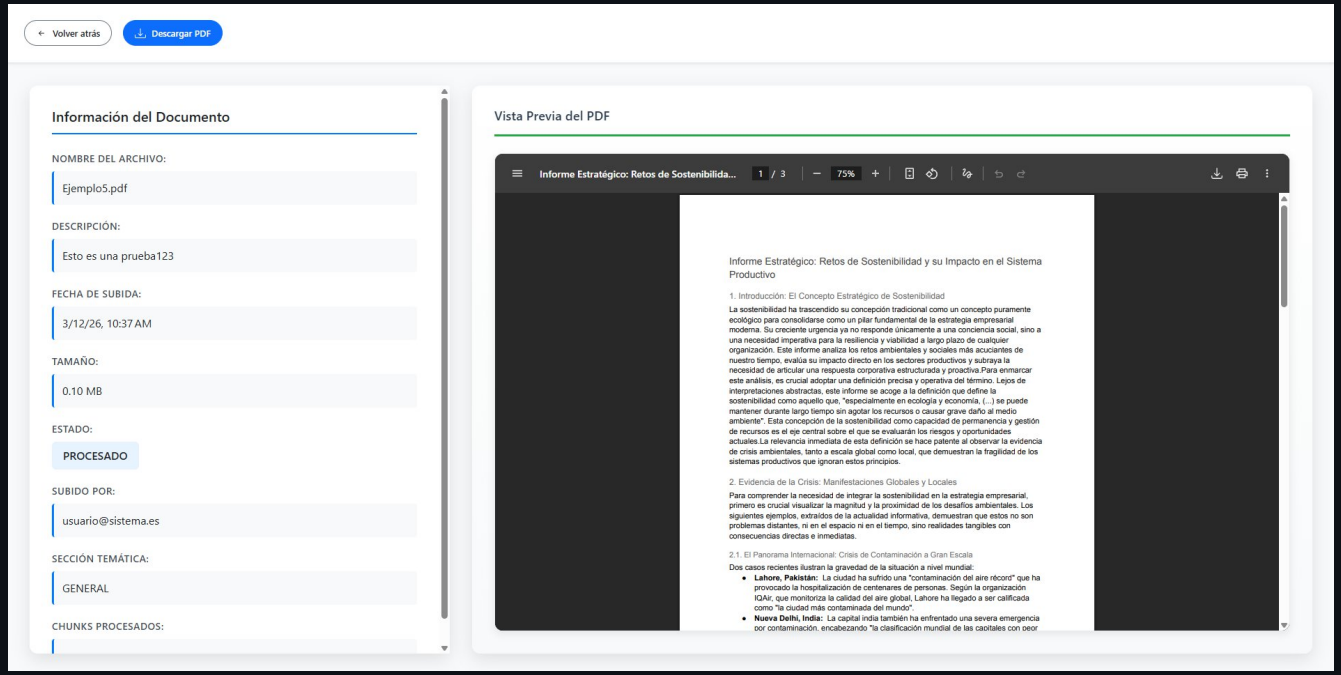
Componente vista previa

FRONTEND DOC 3 / 3

Columna izquierda: metadatos del archivo

Columna derecha: PDF renderizado en iframe

- Accesible a través de boton “Previsualizar”
- Ventana independiente
- Boton volver atrás y descargar



¡Gracias!

por vuestra atención

Moha | Patryk | Richart | Yordan

2º DAW
2026